Rovem — Tech Stack and Mission 1 Plan (provisional)

Oct 12, 2026 · @k._muz

Companion to the Software Requirements Specification, Draft 0.5. Use Git and GitHub, React for the screens, three.js for the 3D scene, a hand-written TypeScript physics module, and Supabase for accounts and saving. The built game is a set of static files served by Nginx on a Linux server, so no Node backend is needed.

Stack

Layer	Choice	Why	Compared with the suggested stack
Version control	Git and GitHub	A check runs the physics tests on every push	Same
Language and build	TypeScript with Vite	Types catch unit mix-ups in the physics; Vite builds the static site	Added
Screens	React	Build, question, results and concepts screens are forms and tables	Same
3D scene	three.js, used directly	It only draws the rover and terrain, so frame rate cannot change results (FR-29). It runs in the browser, not on a backend	Same library, moved to the browser side

Layer	Choice	Why	Compared with the suggested stack
Physics	Own TypeScript module, no physics engine	The SRS requires textbook formulas, a fixed time step and tests against hand calculations (FR-10, FR-29, NFR-4), as set out in the Physics Model Specification. The Build preview calls the same module (FR-3)	Agrees with keeping it simple
Accounts and saving	Supabase: PostgreSQL, sign-in, row-level security	Already decided in the Data Model	Not covered
Account deletion	One Supabase Edge Function	Server-side TypeScript on Deno that can hold the secret key (FR-41). The only application code that runs on a server in the MVP	Replaces the Node backend
Hosting	Nginx on a Linux server in the cloud	Serves the built game files over HTTPS. Later, the same server can run the AI helper's server-side proxy behind Nginx. The cost is upkeep: system updates, the HTTPS certificate and the firewall	Same
Tests	Vitest	Physics cases at 10^{-9} relative error (NFR-4); Knowledge, unlock and merge rules (FR-35, FR-43, FR-44)	Added

Hard-coded movement and gravity

The advice to keep the physics simple and do the calculations yourself is adopted: no physics engine, hand-written formulas, and the numbers on the Build screen worked out by the same code.

Scripted movement is not adopted. The student drives with throttle, brake and steering, and Mission 3 depends on when they brake, so the path cannot be fixed beforehand. The rover is simulated live, in fixed steps.

One correction to the gravity point. Acceleration does depend on gravity once there is friction. Rolling friction and the grip limit both scale with weight, so both change with g:

```
a = (min(F_d,max , μ_g × k_g × m × g) − μ_r × k_r × m × g) ÷ m
  = min(F_d,max ÷ m , μ_g × k_g × g) − μ_r × k_r × g
```

Acceleration is independent of g only when there is no rolling friction and the driving force stays below the grip limit; with no grip at all, a wheeled rover could not move. A heavy rover on Mercury and on Pluto has the same mass but a different weight, so friction and grip differ, which makes it a good future experiment. Planets are future work (SRS Section 6), not part of the MVP.

Mission 1 plan

The prototype is milestones 1 to 8, played as a guest. Accounts (FR-41 to FR-44) follow it. Milestone 2 implements the Physics Model Specification.

#	Milestone	Done when
1	Repository and checks	A TypeScript and Vite project is on GitHub, and one test runs on every push
2	Physics core	The Physics Model Specification is implemented and tests T1 to T10 pass, including all 135 Simulation sheet values to 0.001 s or 0.001 m. Starting build: 100 m in 8.500 s and 85 m in 8.150 s
3	Build screen	Part choice and the Part effects table come from the physics module. The starting build shows 209 kg, 4.59 and 3.61 m/s² acceleration, and 7.37 and 6.08 m/s² braking on asphalt and sand

#	Milestone	Done when
4	Driving scene	A flat three.js scene with keyboard steer, throttle and brake that rise and fall gradually, and a speed and direction display. A stopped rover does not turn, brake beats throttle (FR-6, FR-27), and the same input record gives identical results at 30 and 144 frames per second
5	Mission 1 rules	Two routes, the 9 s limit, the target, pass and fail, and Motor boost and Lighter load side by side on each route, with the timer paused during their questions
6	Questions and results	Three required questions, the time-band run prediction, measured results, and an explanation using distance, time, speed, acceleration and F = ma
7	Local saving	Guest progress saved in local storage in the same shape as the database tables
8	Deployment	Nginx on a Linux server serves the built game over HTTPS, and it opens in a current desktop browser

Sources

- Supabase Edge Functions: server-side TypeScript on a Deno-compatible runtime, with secrets held as project secrets
- Rovem Software Requirements Specification, Draft 0.4, with the Draft 0.5 changes; Data Model; Physics Model Specification
- Rovem Component Values Proposal sheet, for part values and simulation settings